A4 学生版讲义 · 进阶控件 + 管理员端 + 搜索分页 + 服务端监控窗口

> **课程**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元④ **用法**:课前预习、课后复习都能看。内容精炼,配合课堂消化。 **今天**:先从零补三块进阶基础(**进阶控件 / EDT / 发布订阅**),再在跑通的读者端上迭代加管理员端 + 组合搜索,最后给后台服务端加一张"脸"——监控窗口。 **学情提示**:今天用到的 **表格 / 弹窗 / 选项卡进阶控件**、**EDT**、**发布订阅**,单元③都没讲过——本讲义三~五节从零讲,配最小代码,先看懂再动手。

一、迭代式开发:在能跑的系统上加功能

前三个单元,主干链路已经通了:`客户端 → Socket → 服务端 → JDBC → 库`,读者能登录 / 查书 / 借还。今天是在**现有基础上加功能**——这叫**迭代式开发**:不推倒重来,一块一块加。

每加一块永远是三步:**说清需求(一句话 prompt)→ AI 加这一块 → 你跑通 + 验收。**

> AI 生成很快,所以你更要**每加一块就验收一块,别攒着**。前面的服务端、读者端一个字不用改,今天只往上"长"。但动手前,先补三块新东西(三~五节)——不补,今天的界面和监控窗口你看不懂、审不动。

二、⭐ 管理端 = 同一条 C/S 链路换业务

最容易的误区:以为"管理员端"是个全新大工程。其实**它和读者端是同一套链路**,只是业务换了。

维度	读者端(已做)	管理员端(今天)
链路	发一行 → 查库 → 返一行	**完全一样**
命令	`BOOK_QUERY` / `BORROW` / `RETURN`	`BOOK_ADD` / `BOOK_UPDATE` / `BOOK_DELETE` / `READER_LIST` / `BORROW_RECORD`
界面	`ReaderMainFrame` + 查询面板	`AdminMainFrame` + 管理面板 + 编辑对话框
服务端分层	`ClientHandler` → Service → Dao	**同一套,加几个方法**
权限	只能查 / 借还自己的	能增删改图书、看全部借阅统计

结论:管理端不是新技术,是"读者端那条链路换成增删改再走一遍"。今天真正新的是三个进阶控件(三节)——链路你早就会了。所以今天生成更快,你的活主要是**审 CRUD 界面对不对**。

三、Swing 进阶控件零基础速记(没学过表格/弹窗/选项卡也能懂)

管理端三件新事,正好对应三个新控件:

控件	干什么	关键点(记这几句就够)
JTable + DefaultTableModel	把一批数据显示成表格	**数据在 model 里,table 只是画出来**;加行 `model.addRow(...)`、清空 `setRowCount(0)`;取选中行 `getSelectedRow()`(没选返回 -1)+ `getValueAt(行,列)`;**表格只读** = 重写 `isCellEditable` 返回 false
JDialog	弹一个新增/编辑表单	`new JDialog(主窗, 标题, true)` 的 `true` = **模态**(弹窗开着主窗点不了);**传 null = 新增(空表单)、传对象 = 编辑(预填)**;保存按是否 null 调不同接口
JTabbedPane	选项卡分页	`new JTabbedPane()` + `addTab("标题", 面板)`;一个窗口装几块功能,点标签切换
工具栏	一排操作按钮	就是 `JPanel` + `FlowLayout` 排一排按钮,每个 `addActionListener` 绑动作(A3 学过)

> 🔑 一段 mini-CRUD 就能把它们拼齐:**JTable 只读表格 + 工具栏(新增/删除)+ 弹窗表单 + 删除确认**。今天 AI 生成的 `BookManagePanel` 比这多的只有"数据从网络来""表单更漂亮",**骨架一模一样**。

JTable 只读那句一定要认得(审查第一条):

```
new DefaultTableModel(new Object[]{"ID","书名"}, 0) {
    @Override public boolean isCellEditable(int r, int c) { return false; }  //
表格只读
};
```

JDialog 新增/编辑二合一那句:

```
super(owner, book == null ? "新增图书" : "编辑图书", true);  // modal
if (book != null) preload(book);                              // 编辑时预填
```

四、EDT 零基础速记(没学过多线程碰界面也能懂)

事实:Swing 界面的所有显示和刷新,都由一条专门的线程负责——**EDT(Event Dispatch Thread,事件分发线程)**。

词	是什么	生活类比
EDT	Swing 专门刷界面的那条线程	一块黑板,只准**值日生**上去写
铁律	所有界面修改**只能在 EDT 上做**	别人冲上去写黑板就乱了

> 🔑**为什么?Swing 不是线程安全的。** 两个线程同时改同一个界面组件会乱套,还特别阴——大多数时候"看着没事",偶尔才出诡异 bug、极难复现。所以 Swing 干脆定死:界面只让 EDT 一个线程碰。

什么时候你"不在 EDT 上"(两种,都要切回 EDT):

- ① **后台耗时任务**(单元③):网络请求丢到后台线程,后台线程拿到结果要显示时,不在 EDT。
- ② **别的线程的回调 / 推送**(今天监控窗口):日志是各个客户端处理线程产生的,它们调 `onLog(...)` 时根本不在 EDT。

解法:用 `SwingUtilities.invokeLater(() -> { 改界面 })` 把"改界面"排队交给 EDT:

```
new Thread(() -> {                              // 后台线程,不是 EDT
    SwingUtilities.invokeLater(() -> label.setText("计数:" + n));  // 切回 EDT 才安
全
}).start();
```

> 🔑一句话:**别的线程不许碰界面,要碰先 `invokeLater` 回到 EDT。** 这和单元③"耗时请求放后台不卡界面"是**同一道理的两面**:那边"别在 EDT 干耗时活",这边"别在非 EDT 改界面"。**EDT 只做刷界面这一件事。**

五、发布 - 订阅零基础速记(广播站 / 订阅号)

难题:监控窗口的日志和在线数**到处产生**(各个客户端线程),却要显示到**一个**窗口。让每个线程都直接持有窗口去改它,耦合太死。

解法:加个"中间人"——**发布 - 订阅(观察者)模式**:

- **广播站/订阅号(发布者)**:只管"喊一声事件发生了",**不关心谁在听**。
- **听众/粉丝(订阅者)**:只管"听到就处理",不关心事件从哪来。
- 中间的"广播站"把两边接上——**发布者和订阅者解耦了**。

三个部件(最小模型):

部件	干什么
订阅接口 Listener	约定订阅者能收什么事件,如 `onEvent(String)`
发布者 / 中枢	持有一个订阅者;`publish(...)` 先记一笔、再通知订阅者(为 null 就跳过)
订阅者	实现 Listener,`setListener(this)` 注册,事件来了 `onEvent` 被调到

> 🔑 细节:没订阅者时 `publish` 照样能"记录"、只是不通知——正好对应"**监控窗口没开时,日志照样打到控制台**"。

对应到今天:**ServerContext(中枢)/ ServerContext.Listener(接口)/ ServerMonitorFrame(订阅者)**。

> 🔑 **在线计数为什么用 `AtomicInteger`?** 在线数是"多个客户端线程同时 ±1"的,普通 `int online++` 在多线程下会数错。用 `AtomicInteger` 的 `incrementAndGet()` / `decrementAndGet()`,并发下每次 ±1 都不出错。记住"**并发 ±1 用原子类**"。

六、组合搜索:动态拼 SQL + 防注入

图书查询按**关键词(书名 / 作者模糊)+ 分类**组合筛。核心在服务端(答案版 `BookDao.query`):

```
sql = "... WHERE 1 = 1 ";
if (有关键词) sql += "AND (b.title LIKE ? OR b.author LIKE ?) ";  // 占位符,不是拼接
if (有分类)   sql += "AND b.category_id = ? ";
```

两个要点:

- **`WHERE 1 = 1` + 按条件 append**:有就加、没就不加,SQL 永远合法。
- **`LIKE ?` 用占位符**:`%关键字%` 作为**参数**传,`PreparedStatement` 防注入(单元① PRD 的规范落地)。

分页(可选):数据多时**不要一次全查回来**,后端只查这一页——SQL 加 `LIMIT ? OFFSET ?`(offset=(page-1)*size),再 `SELECT COUNT()` 查总数;客户端记住 page,翻页改 page` 重查,总页数 = ceil(总数/size)。

七、⭐ 服务端可视化:让后台服务端看得见

7.1 问题(为什么要监控窗口)

我们的服务端是**后台应用服务器,本身没有界面**。打成 jar 双击运行,没有控制台、界面上什么也没有,像"没起来"。端口开没开、几个客户端、谁登录了——全是黑盒。

7.2 解法:监控窗口 + 发布订阅(怎么做)

给服务端加一个 Swing 监控窗口(答案版 `ServerMonitorFrame`):**监听端口 + 运行状态 + 在线客户端数 + 实时日志 + 启停按钮**(顶部几个 JLabel、中间只读 JTextArea、底部两个按钮——都是三节学过的控件);它是服务端 jar 的默认入口(仍保留控制台模式)。

难点:日志和在线数由**各个客户端处理线程**产生,却要显示到**一个**窗口。用一个 `ServerContext` 做中枢——**发布 - 订阅**(五节):

```
ClientHandler(各线程) ──log()/clientConnected()/clientDisconnected()──►
ServerContext
                                                          │(集中日志 + AtomicInteger
计数)
                                                          ▼ onLog / onOnlineChange
                                                  ServerMonitorFrame(订阅者,显示)
```

> "谁产生事件"和"谁显示事件"**解耦**了:线程只管喊一声,窗口只管听到就显示。没有窗口时(控制台模式),日志照样打到 `System.out`。

7.3 ⭐ 铁律:所有 UI 更新必须切回 EDT

`onLog` / `onOnlineChange` 是被**客户端处理线程**调用的(不是 EDT,见四节)。解法——投递回 EDT:

```
@Override public void onLog(String line) {
    SwingUtilities.invokeLater(() -> {            // 切回 EDT
        logArea.append(line + "\n");
        logArea.setCaretPosition(logArea.getDocument().getLength());
    });
}
```

顺带:服务端要能启停,`LibraryServer` 拆成 `start()`(开后台 accept 线程)/ `stop()`(`serverSocket.close()` 让阻塞的 `accept()` 抛异常退出);`main` 保留作控制台入口。

7.4 日志脱敏(第二条铁律)

日志会显示给人看、可能落盘。**绝不能把 `LOGIN|admin|123456` 整行记进去**——密码就泄露了。答案版只记 `|` 前的命令名 + 结果:

```
String cmd = line.contains("|") ? line.substring(0, line.indexOf('|')) : line;
ServerContext.log(addr + "  " + cmd + " → " + (response.startsWith("OK") ? "OK" :
"ERR"));
```

日志里看到的是 `LOGIN → OK`,**不是** `LOGIN|admin|123456`。既是隐私,也是安全底线。

八、两段 prompt 速查(可复制)

① 生成管理员端

```
基于现有代码,做管理员端主窗(选项卡):
- 图书管理:JTable + 工具栏(新增/编辑/删除/刷新/搜索),新增/编辑用 JDialog 表单,协议
BOOK_ADD|json、BOOK_UPDATE|json、BOOK_DELETE|id(服务端走 BookDao),删除有确认框;
- 分类管理、读者管理(READER_LIST)、借阅记录/超期统计(BORROW_RECORD)各一个选项卡。
只加管理员端,别动读者端。
```

② 生成服务端监控窗口

```
给服务端加一个监控窗口 ServerMonitorFrame(Swing),作为服务端 jar 的启动入口:
- 显示监听端口、运行状态、在线客户端数、实时日志(每次客户端接入/登录/借书打一行)、启动/停止按
钮;
- 加一个 ServerContext(集中日志 + 在线计数,发布订阅),ClientHandler 接入/断开时在线数±1、
每条请求记"命令→OK/ERR"(不记密码);
- LibraryServer 改成可 start()/stop()。保留一个控制台 main 便于无界面运行。
```

九、审查清单(今天你要盯的)

管理端三条:

- [ ] 表格只读(`isCellEditable` 返回 false)
- [ ] 删除有确认框(`showConfirmDialog`)
- [ ] 请求走异步(`Async.run`,不在界面线程/EDT 发网络请求)

监控窗口两条铁律:

- [ ] 🔴 跨线程更新 UI:`onLog` / `onOnlineChange` 里改界面包在 `SwingUtilities.invokeLater`
- [ ] 🔴 日志不记密码:请求日志只记 `|` 前的命令名 + OK/ERR,不记参数

发现不对,用**纠错三步法**(定位 → 描述 → 只改一处)让 AI 补。**让 AI 干活、你把关,这就是"信任但验证"。**

十、FAQ

Q1:管理员端是不是要重新学一大堆新东西? 不是。管理端和读者端是**同一条 C/S 链路换业务**——都是"JTable 展示 + Async 异步请求 + 服务端 Handler→Service→Dao",界面上新东西只有一个 JDialog 表单。主干打通后,加功能是复制,不是重造。

Q2:JTable 和 DefaultTableModel 什么关系? `DefaultTableModel` **装数据**(表头 + 每行内容),`JTable` **把 model 画成界面表格**。加行、清空、取值都操作 model,不操作 JTable。取选中行两步走:`getSelectedRow()` 拿行号,`getValueAt(行,列)` 拿值。

Q3:为什么表格要设成只读? 数据不该在表格格子里直接乱改(容易误操作、也没经过校验)。改数据统一走"选中 → 点编辑 → 弹表单 → 保存",可控。所以 `DefaultTableModel` 重写 `isCellEditable` 返回 false。

Q4:JDialog 怎么做到"一个对话框既新增又编辑"? 传进来一个对象:传 `null` = 新增(表单空着),传一个已有的书 = 编辑(字段预填进输入框);保存时判断原来是不是 null,调"新增"或"更新"接口。构造时第三个参数 `true` = 模态,弹窗开着主窗点不了。

Q5:什么是 EDT?为什么别的线程不能直接改界面? EDT 是 Swing 专门刷界面的那个线程。Swing 不是线程安全的,多个线程同时碰界面会出诡异 bug(错乱、偶发卡死)。所以规定:**界面只能在 EDT 上改**;别的线程要改,用 `SwingUtilities.invokeLater(...)` 把动作投递回 EDT。

Q6:发布订阅到底解决了什么? 解决"到处产生的事件,怎么汇到一个窗口"。让每个线程直接改窗口耦合太死;加一个中枢 `ServerContext`,线程只管 `log()` / `clientConnected()` "喊一声",窗口订阅它、听到就显示。发布者和订阅者解耦,以后消息队列、事件监听到处都是这个模式。

Q7:服务端明明能跑,为什么还要个监控窗口? 因为它是后台程序、本身没界面,双击 jar"看不到任何东西",运维不知道它在不在跑、有没有人连。监控窗口让它"把自己在干什么说出来"——端口、在线数、日志一目了然。这在答辩里是个很出彩的点。

Q8:日志为什么不能记密码? 日志会显示给人看、可能写到文件里长期留存。如果把 `LOGIN|admin|123456` 整行记进去,密码就泄露了。答案版只记 `|` 前的命令名(`LOGIN`)+ 结果(OK/ERR),参数一律不记。这既是隐私,也是安全底线。

Q9:监控窗口生成后,我主要审查什么? 两条铁律:① 界面更新有没有切 EDT(`invokeLater`);② 日志有没有记密码。AI 图快很容易漏——你能看出来、能让它补,就是你的价值。

十一、⭐ 管理端四大功能族(E①–E④)· prompt 速查 + 审查要点速记

> 前面 Part 6–8 打通了管理端主干(图书 CRUD + 组合搜索 + 监控窗口)。真正"能运营的后台"还差四块:**统计报表、操作日志审计、用户审核治理、分类管理。 和前面最大的不同:这四块 demo 工程里没有答案版代码**,全程走"纯 prompt 模式"——你靠 prompt 说清、靠审查把关、靠真实数据验证。**没有标准答案可抄,这正是你毕业后的真实工作。**

四族一览(先记住每块的"灵魂审查点")

功能族	命令 / 产物	灵魂审查点
E① 借阅统计报表	`STATS_BORROW_BY_CATEGORY` / `STATS_HOT_BOOKS` / `STATS_STOCK_ALERT` + `StatisticsPanel`	**审 `GROUP BY` 聚合 SQL 口径**(报表最会骗人)
E② 操作日志审计	补建 `t_operation_log` + `LOG_LIST` + `OperationLogPanel`	**异步不阻塞 + 不记密码 + 日志不进主事务**
E③ 用户审核治理	`USER_APPROVE` / `USER_DISABLE` + `t_user.status` 三态	**"能改状态 ≠ 状态起作用"的禁用漏洞**
E④ 分类 CRUD	`CATEGORY_ADD/UPDATE/DELETE` + `CategoryManagePanel`	**删除关联拦截:删了谁成孤儿**

四段 prompt 速查(可复制)

① 借阅统计报表

```
基于现有 C/S 图书馆代码,新增管理端「借阅统计报表」,只加统计相关。
一、服务端三条协议命令(dao 层 PreparedStatement,响应 OK|json):
  1. STATS_BORROW_BY_CATEGORY|start_date|end_date —— 各分类区间内借阅次数,
     GROUP BY c.id,c.name,ORDER BY 次数 DESC。返回 [{category_name,borrow_count}]
  2. STATS_HOT_BOOKS|limit —— GROUP BY b.id,b.title,ORDER BY 次数 DESC LIMIT ?。
     返回 [{title,author,borrow_count}]
  3. STATS_STOCK_ALERT|threshold —— WHERE available_stock < ? ORDER BY
available_stock ASC(不用 GROUP BY)。
     返回 [{title,available_stock,total_stock}]
二、客户端 StatisticsPanel(AdminMainFrame 新选项卡「统计报表」):日期/阈值输入 + 刷新,
  三个 JTable 分区展示,库存预警高亮低库存行,查询走 Async.run。
约束:PreparedStatement 占位符,不许把日期/阈值拼进 SQL;空结果显示"暂无数据"。
```

② 操作日志审计

```
基于现有 C/S 图书馆代码,新增管理端「操作日志审计」,只加日志相关。
一、补建表 t_operation_log(id,user_id,user_name,operation_type,operation_desc,
   target_table,target_id,ip_address,created_at)——A2 建库没建这张,这里补。
二、关键操作成功后记一条(登录/借/还/续借/图书增删改/用户审核禁用):
  - 只记操作类型 + 人话描述,绝不把密码/完整请求行写进 operation_desc;
  - 独立线程池异步落库,不阻塞主业务,写失败只告警;
  - 绝不能放进借书/还书事务里(避免日志失败连累库存回滚)。
三、查询 LOG_LIST|start_date|end_date|page|size:
  WHERE created_at BETWEEN ? AND ? ORDER BY created_at DESC LIMIT ? OFFSET ?,另查
COUNT(*)。
四、客户端 OperationLogPanel(选项卡「操作日志」):日期区间 + 查询 + 翻页,
  JTable 列 时间|操作人|类型|描述|IP,走 Async.run。
约束:PreparedStatement;operation_desc 不许出现任何密码。
```

③ 用户审核治理

```
基于现有 C/S 图书馆代码,新增管理端「用户审核治理」,接上现有登录/注册链路。
一、t_user.status 三态 ENUM('pending','active','disabled') 默认 'pending'(没有该列先
ALTER 补)。
二、服务端命令:
  1. USER_APPROVE|user_id —— UPDATE t_user SET status='active' WHERE id=? AND
status='pending'。
  2. USER_DISABLE|user_id —— UPDATE t_user SET status='disabled' WHERE id=?;不允许
禁用管理员自己。
  3. USER_LIST|status|page|size —— 按状态筛(status 空=全部)。
三、⭐ 改 A2 的 LOGIN:只有 status='active' 才允许登录;pending→ERR|账号待审
核,disabled→ERR|账号已被禁用。
四、⭐ 改 A2 的 REGISTER:新用户 status 默认 'pending',提示"注册成功,请等待管理员审核"。
五、客户端:用户管理面板加状态列 + 状态筛选下拉 + 选中后"审核通过""禁用"两按钮(走 Async)。
约束:PreparedStatement;审核/禁用 UPDATE 必须带 WHERE id=?,绝不漏 WHERE 批量误改。
```

④ 分类 CRUD

```
基于现有代码,给管理端加「分类管理」CategoryManagePanel(照 BookManagePanel 套路,换分类业务),
加进 AdminMainFrame 新选项卡「分类管理」。只改分类相关。
服务端命令:
- CATEGORY_LIST —— 列出所有分类(id/name/description/该分类下图书数量)。
- CATEGORY_ADD|name|description —— name 唯一,重名返回 ERR|分类名已存在。
- CATEGORY_UPDATE|id|name|description。
- CATEGORY_DELETE|id —— ⭐ 删前 SELECT COUNT(*) FROM t_book WHERE category_id=?,
  有图书则拒绝返回 ERR|该分类下还有N本图书,不能删除;无图书才真正删。
客户端:JTable(分类名/描述/图书数量)+ 工具栏(新增/编辑/删除/刷新),
新增/编辑用 JDialog 表单,删除有确认框,走 Async。
约束:PreparedStatement;删除必须先做关联检查再删。
```

各族审查要点速记(点点按钮测不出来,必须去读代码/撞真实数据)

E① 统计报表 —— 审 `GROUP BY` 聚合口径

- [ ] 🔴 `GROUP BY` 字段完整:`SELECT` 里非聚合列都要进 `GROUP BY`(如 `GROUP BY c.id, c.name`,只写 `c.name` 会把同名分类并成一行)
- [ ] 🔴 `COUNT` 数对东西:数借阅记录用 `COUNT(br.id)`;`COUNT(*)` 在 LEFT JOIN 下会把"没借阅的分类"也数成 1
- [ ] 🔴 排序/阈值方向:热门榜 `DESC`、库存预警 `ASC` 且阈值 `< threshold`(写成 `>` 列出的全是库存充足的书)
- [ ] 🟡 时间区间用占位符 `BETWEEN ? AND ?`,不拼字符串
- 🎬 用真实数据点撞:手数种子库某分类真实借阅次数,对不上就是口径错(如 JOIN 重复计数翻倍)

E② 操作日志审计 —— 三条底线

- [ ] 🔴 不记密码:`operation_desc` 只记操作类型 + 人话("用户 admin 登录成功"),绝无明文密码
- [ ] 🔴 异步不阻塞:独立线程池落库,写日志失败只告警、不连累主业务返回
- [ ] 🔴 日志不进主事务:INSERT 审计日志写在借书/还书事务**提交之后**,别夹在事务里(否则日志失败会让借书回滚)
- [ ] 🟡 覆盖关键操作:登录/借/还/增删改/审核都要有留痕
- 💡 与监控窗口内存日志的区别:**审计=数据库持久化、可按时间/用户回溯;监控窗=内存实时状态、关掉就没**

E③ 用户审核治理 —— 三条安全线

- [ ] 🔴 注册默认 `pending`:REGISTER 插入用户 status 默认待审
- [ ] 🔴 未审核不能登录:LOGIN 加 `status='active'` 校验,pending/disabled 分别返回不同 ERR
- [ ] 🔴 禁用即时生效:禁用后新登录请求即时读最新 status,不能有缓存放行
- [ ] 🟡 审核/禁用 UPDATE 必须带 `WHERE id=?`,绝不批量误改
- ⚠️ 头号坑:AI 常只做了"改 status"却忘了在登录处校验 → **禁用形同虚设**。"能改状态 ≠ 状态真的起作用"

E④ 分类 CRUD —— 关联完整性

- [ ] 🔴 删除关联拦截:`CATEGORY_DELETE` 删前 `SELECT COUNT(*) FROM t_book WHERE category_id=?`,`>0` 就拒绝
- [ ] 🟡 name 唯一冲突返回友好 ERR,不直接抛 SQL 异常给用户
- [ ] 🟡 表格只读 / 删除确认 / 异步(同图书管理三条)

- 🔑 通用心法:**删任何"被别人依赖"的东西前,先问"删了它谁会成孤儿"**(分类被图书依赖、用户被借阅记录依赖……到处都是这个模式)

> 四块合起来练的是**同一件事的四种样子**:AI 能很快把功能"做出来",但"做对"要靠你审。报表的口径、日志的边界、禁用的门禁、删除的关联,这些坑 AI 图快都爱踩——而它们又都是点点按钮测不出来、必须读代码/撞真实数据才发现的。**这就是"范式 2"里你最值钱的能力:不是写代码,是判断 AI 写的对不对。**

今日产出

从零补上**三块进阶基础**:Swing 进阶控件(`JTable` + `DefaultTableModel` / `JDialog` / `JTabbedPane` / 工具栏)、**EDT**(谁能碰界面)、**发布 - 订阅**(广播站 / 订阅号);在跑通的读者端上**迭代**出**管理员端**(图书 CRUD / 读者 / 超期统计,认清"管理端 = 换业务")、**组合搜索**(`WHERE 1=1` 动态拼 + `LIKE ?` 防注入),并给后台服务端加了**监控窗口**(发布订阅 + 集中日志 + 在线计数 + **UI 切 EDT** + 可 start/stop)——**系统从"读者能用"长成了"管理员能管、后台看得见",关键处都由你审查把关。**